

```

import React, {
  forwardRef,
  useState,
  useEffect,
  useCallback,
  useMemo,
} from "react";
import { Loader2, LucideIcon } from "lucide-react";
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

// 🌀 LOCAL VANILLA CN UTILITY CORES
export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

// ① LOCAL TYPE ISOLATION GATEWAY
export interface PremiumButtonProps extends
  React.ButtonHTMLAttributes<HTMLButtonElement> {
  variant?:
    | "brand"
    | "secondary"
    | "outline"
    | "ghost"
    | "destructive"
    | "link"
    | "premium";
  size?: "xs" | "sm" | "md" | "lg" | "xl" | "icon";
  isLoading?: boolean;
  loadingText?: string;
  leftIcon?: LucideIcon;
  rightIcon?: LucideIcon;
  iconOnly?: boolean;
  fullWidth?: boolean;
  isDisabled?: boolean;
  asChild?: boolean;
  loadingPosition?: "left" | "right" | "center";
}

const variantStyles = {
  brand:
    "bg-brand-primary text-white hover:bg-brand-hover active:bg-brand-primary
active:scale-[0.98] shadow-sm hover:shadow-md transition-[var(--transition-fast)]",
  secondary:
    "bg-secondary text-secondary-foreground hover:bg-secondary/80
active:bg-secondary active:scale-[0.98] border border-border transition-all
duration-200 ease-out",
  outline:
    "border border-input bg-transparent hover:bg-accent hover:text-accent-foreground
active:bg-accent active:scale-[0.98] transition-all duration-200 ease-out",

```

```

ghost:
  "bg-transparent hover:bg-accent hover:text-accent-foreground active:bg-accent
active:scale-[0.98] transition-all duration-200 ease-out",
destructive:
  "bg-destructive text-destructive-foreground hover:bg-destructive/90
active:bg-destructive active:scale-[0.98] shadow-sm hover:shadow-md transition-all
duration-200 ease-out",
link: "text-brand-primary underline-offset-4 hover:underline text-sm font-medium
transition-colors duration-200 ease-out",
premium:
  "bg-gradient-to-r from-brand-primary via-brand-primary/90 to-brand-primary/80
text-white hover:from-brand-hover hover:via-brand-primary/80
hover:to-brand-primary/70 active:from-brand-primary active:via-brand-primary/90
active:to-brand-primary active:scale-[0.98] shadow-lg hover:shadow-xl
active:shadow-lg transition-[var(--transition-fast)] relative overflow-hidden",
};

```

```

const sizeStyles = {
  xs: "h-7 px-2.5 text-xs gap-1",
  sm: "h-8 px-3 text-sm gap-1.5",
  md: "h-10 px-4 py-2 text-base gap-2",
  lg: "h-12 px-6 text-lg gap-2.5",
  xl: "h-14 px-8 text-xl gap-3",
  icon: "h-10 w-10 p-0",
};

```

```

const iconSizeStyles = {
  xs: "w-3 h-3",
  sm: "w-3.5 h-3.5",
  md: "w-4 h-4",
  lg: "w-5 h-5",
  xl: "w-6 h-6",
  icon: "w-5 h-5",
};

```

```

const loadingSizeStyles = {
  xs: "w-3 h-3",
  sm: "w-4 h-4",
  md: "w-5 h-5",
  lg: "w-6 h-6",
  xl: "w-7 h-7",
  icon: "w-5 h-5",
};

```

```

const focusStyles =
  "focus:outline-none focus-visible:ring-2 focus-visible:ring-ring
focus-visible:ring-offset-2 focus-visible:ring-offset-background";

```

```

const disabledStyles =
  "disabled:pointer-events-none disabled:opacity-50 disabled:cursor-not-allowed";

```

```

const baseStyles =
  "inline-flex items-center justify-center font-sans font-medium leading-none
rounded-surface transition-all duration-200 ease-out select-none";

export const PremiumButton = forwardRef<HTMLButtonElement, PremiumButtonProps>(
  (
    {
      variant = "brand",
      size = "md",
      isLoading = false,
      loadingText,
      leftIcon: LeftIcon,
      rightIcon: RightIcon,
      iconOnly = false,
      fullWidth = false,
      isDisabled = false,
      asChild = false,
      loadingPosition = "left",
      className,
      children,
      disabled,
      ...props
    },
    ref,
  ) => {
    const isActuallyDisabled = disabled || isDisabled || isLoading;
    const hasLeftIcon = Boolean(LeftIcon) && !isLoading;
    const hasRightIcon = Boolean(RightIcon) && !isLoading;
    const showLoadingSpinner =
      isLoading && (loadingPosition !== "center" || !loadingText);
    const showLoadingText = isLoading && loadingText;

    const iconSize = useMemo(() => iconSizeStyles[size], [size]);
    const loadingSize = useMemo(() => loadingSizeStyles[size], [size]);

    // Type-safe icon components
    const LeftIconComponent = LeftIcon as
      | React.ComponentType<React.SVGProps<SVGSVGElement>>
      | undefined;
    const RightIconComponent = RightIcon as
      | React.ComponentType<React.SVGProps<SVGSVGElement>>
      | undefined;

    // Extract asChild prop to avoid TypeScript error
    const { as: AsComponent } = props as { as?: React.ElementType } & Omit<
      typeof props,
      "as"
    >;
  }
);

```

```

// Use typed components for rendering to satisfy TypeScript
const RenderLeftIcon = LeftIconComponent;
const RenderRightIcon = RightIconComponent;

const baseClasses = useMemo(
  () =>
    cn(
      baseStyles,
      variantStyles[variant],
      sizeStyles[size],
      focusStyles,
      disabledStyles,
      fullWidth && "w-full",
      iconOnly && "p-0",
      className,
    ),
  [variant, size, fullWidth, iconOnly, className],
);

const [isPressed, setIsPressed] = useState(false);
const [isFocusVisible, setIsFocusVisible] = useState(false);
const [ripple, setRipple] = useState<{
  x: number;
  y: number;
  size: number;
} | null>(null);

const handleMouseDown = useCallback(
  (e: React.MouseEvent<HTMLButtonElement>) => {
    if (isActuallyDisabled) return;
    const rect = e.currentTarget.getBoundingClientRect();
    const size = Math.max(rect.width, rect.height);
    setRipple({
      x: e.clientX - rect.left - size / 2,
      y: e.clientY - rect.top - size / 2,
      size,
    });
    setIsPressed(true);
  },
  [isActuallyDisabled],
);

const handleMouseUp = useCallback(() => {
  setIsPressed(false);
  setTimeout(() => setRipple(null), 300);
}, []);

const handleMouseLeave = useCallback(() => {
  setIsPressed(false);
}, []);

```

```

const handleFocus = useCallback(
  (e: React.FocusEvent<HTMLButtonElement>) => {
    if (e.nativeEvent.target === e.currentTarget) {
      setIsFocusVisible(true);
    }
  },
  [],
);

```

```

const handleBlur = useCallback(() => {
  setIsFocusVisible(false);
  setIsPressed(false);
}, []);

```

```

const handleKeyDown = useCallback(
  (e: React.KeyboardEvent<HTMLButtonElement>) => {
    if (e.key === " " || e.key === "Enter") {
      e.preventDefault();
      setIsPressed(true);
    }
  },
  [],
);

```

```

const handleKeyUp = useCallback(
  (e: React.KeyboardEvent<HTMLButtonElement>) => {
    if (e.key === " " || e.key === "Enter") {
      setIsPressed(false);
    }
  },
  [],
);

```

```

useEffect(() => {
  if (isLoading) {
    setIsPressed(false);
  }
}, [isLoading]);

```

```

const rippleStyles = useMemo(
  () =>
    ripple
      ? {
          width: ripple.size,
          height: ripple.size,
          left: ripple.x,
          top: ripple.y,
        }
      : {},
);

```

```

    [ripple],
  );

const content = useMemo(() => {
  if (showLoadingText) {
    return (
      <span className="flex items-center justify-center gap-2">
        {loadingPosition === "left" && (
          <Loader2
            className={cn(loadingSize, "animate-spin")}
            aria-hidden="true"
          />
        )}
        <span>{loadingText}</span>
        {loadingPosition === "right" && (
          <Loader2
            className={cn(loadingSize, "animate-spin")}
            aria-hidden="true"
          />
        )}
      </span>
    );
  }

  if (showLoadingSpinner && loadingPosition === "center") {
    return (
      <Loader2
        className={cn(loadingSize, "animate-spin")}
        aria-hidden="true"
      />
    );
  }

  return (
    <span
      className={cn(
        "flex items-center justify-center gap-1.5",
        iconOnly && "justify-center",
      )}
    >
      {showLoadingSpinner && loadingPosition === "left" && (
        <Loader2
          className={cn(loadingSize, "animate-spin")}
          aria-hidden="true"
        />
      )}
      {hasLeftIcon && RenderLeftIcon && (
        <RenderLeftIcon className={iconSize} aria-hidden="true" />
      )}
      {!iconOnly && children}
    </span>
  );
});

```

```

    {hasRightIcon && RenderRightIcon && (
      <RenderRightIcon className={iconSize} aria-hidden="true" />
    )}
    {showLoadingSpinner && loadingPosition === "right" && (
      <Loader2
        className={cn(loadingSize, "animate-spin")}
        aria-hidden="true"
      />
    )}
  </span>
);
}, [
  showLoadingText,
  loadingText,
  loadingPosition,
  showLoadingSpinner,
  loadingSize,
  iconOnly,
  hasLeftIcon,
  LeftIcon,
  children,
  hasRightIcon,
  RightIcon,
  iconSize,
]);

const premiumGlow = useMemo(() => {
  if (variant !== "premium" || isActuallyDisabled) return null;
  return (
    <div
      className="absolute inset-0 bg-gradient-to-r from-brand-primary/20
via-brand-primary/10 to-brand-hover/20 opacity-0 hover:opacity-100
transition-opacity duration-300 pointer-events-none rounded-surface"
      aria-hidden="true"
    />
  );
}, [variant, isActuallyDisabled]);

const pressedOverlay = useMemo(() => {
  if (!isPressed || isActuallyDisabled) return null;
  return (
    <div
      className="absolute inset-0 bg-black/10 pointer-events-none
rounded-surface"
      aria-hidden="true"
    />
  );
}, [isPressed, isActuallyDisabled]);

const focusRing = useMemo(() => {

```

```

    if (!isFocusVisible || isActuallyDisabled) return null;
    return (
      <div
        className="absolute inset-0 ring-2 ring-ring ring-offset-2
ring-offset-background pointer-events-none rounded-surface"
        aria-hidden="true"
      />
    );
  }, [isFocusVisible, isActuallyDisabled]);

const buttonContent = (
  <>
    {ripple && (
      <span
        className="absolute rounded-full bg-current opacity-10
pointer-events-none animate-ripple"
        style={rippleStyles}
        aria-hidden="true"
      />
    )}
    {premiumGlow}
    {pressedOverlay}
    {focusRing}
    <span className="relative z-10 flex items-center justify-center w-full">
      {content}
    </span>
  </>
);

if (asChild) {
  const ChildComponent = AsComponent || "button";
  return (
    <ChildComponent
      ref={ref}
      className={baseClasses}
      disabled={isActuallyDisabled}
      aria-disabled={isActuallyDisabled}
      aria-busy={isLoading}
      aria-label={iconOnly ? (children as string) : undefined}
      onMouseDown={handleMouseDown}
      onMouseUp={handleMouseUp}
      onMouseLeave={handleMouseLeave}
      onFocus={handleFocus}
      onBlur={handleBlur}
      onKeyDown={handleKeyDown}
      onKeyUp={handleKeyUp}
      {...props}
    >
      {buttonContent}
    </ChildComponent>
  );
}

```



```

    );
  }

  return (
    <button
      ref={ref}
      className={baseClasses}
      disabled={isActuallyDisabled}
      aria-disabled={isActuallyDisabled}
      aria-busy={isLoading}
      aria-label={iconOnly ? (children as string) : undefined}
      onMouseDown={handleMouseDown}
      onMouseUp={handleMouseUp}
      onMouseLeave={handleMouseLeave}
      onFocus={handleFocus}
      onBlur={handleBlur}
      onKeyDown={handleKeyDown}
      onKeyUp={handleKeyUp}
      {...props}
    >
      {buttonContent}
    </button>
  );
},
);

PremiumButton.displayName = "PremiumButton";

// 🌀 PREMIUM BUTTON VARIANTS EXPORT
export const premiumButtonVariants = {
  brand: variantStyles.brand,
  secondary: variantStyles.secondary,
  outline: variantStyles.outline,
  ghost: variantStyles.ghost,
  destructive: variantStyles.destructive,
  link: variantStyles.link,
  premium: variantStyles.premium,
} as const;

export const premiumButtonSizes = {
  xs: sizeStyles.xs,
  sm: sizeStyles.sm,
  md: sizeStyles.md,
  lg: sizeStyles.lg,
  xl: sizeStyles.xl,
  icon: sizeStyles.icon,
} as const;

// 🌀 CONVENIENCE EXPORTS
export type PremiumButtonVariant = keyof typeof premiumButtonVariants;

```

```
export type PremiumButtonSize = keyof typeof premiumButtonSizes;
```